

# PicoloDSP

## UI Kit & Implementation Manual

Open DSP. Native Android.

Este documento transforma o conceito visual do PicoloDSP em um sistema de interface implementável para Android nativo. A proposta prioriza uso em palco, leitura rápida, baixa carga cognitiva e separação clara entre UI e engine DSP em C++.

### 1. Princípios de produto

- Performance first: a UI nunca deve bloquear a thread de áudio; mudanças visuais e de estado devem atravessar uma camada de controle desacoplada do callback DSP.
- Stage friendly: controles grandes, contraste alto, estados ativos óbvios e interações possíveis com uma mão.
- Signal-chain centric: a cadeia de áudio é a navegação principal. O usuário vê rapidamente INPUT -> FX -> NAM -> IR -> FX -> OUTPUT.
- Hardware feel, software flexibility: controles lembram equipamento musical, mas preservam padrões Android para navegação, acessibilidade e estados.
- Open-source identity: a estética deve comunicar ferramenta técnica, comunidade e transparência, sem parecer clone visual de um fabricante.

### 2. Design tokens

| Token          | Valor   | Uso                 | Observação                                        |
|----------------|---------|---------------------|---------------------------------------------------|
| bg/base        | #090D10 | Fundo principal     | OLED-friendly, evita preto absoluto em toda a UI. |
| surface/1      | #11171C | Cards e barras      | Superfície padrão.                                |
| surface/2      | #171F25 | Inputs e controles  | Diferencia elementos interativos.                 |
| text/primary   | #F4F7F9 | Texto principal     | Contraste alto.                                   |
| text/secondary | #9AA6AE | Labels secundárias  | Evitar abaixo de 14sp em palco.                   |
| accent/orange  | #FF6A1A | Brand, NAM, seleção | Cor primária PicoloDSP.                           |
| accent/teal    | #23D6C5 | Drive / connected   | Estados ativos e DSP algorítmico.                 |
| accent/yellow  | #FFC43D | IR / cabinet        | Convolução e cabinet.                             |
| accent/purple  | #A75AF2 | EQ / modulation     | Processors de shaping/modulação.                  |
| accent/blue    | #3C9BFF | Delay / reverb      | Efeitos temporais.                                |
| state/error    | #FF4D5E | Erro / clip         | Usar somente para alerta real.                    |

A paleta abaixo define cores semânticas por família de processamento. Tipografia e iconografia possuem especificações próprias nas seções 2.1 e 2.2.

## 2.1 Tipografia

**Famílias recomendadas:** Space Grotesk para títulos e comunicação de marca; Inter para a interface operacional; JetBrains Mono apenas para telemetria e valores técnicos. A combinação mantém a linguagem técnica sem sacrificar leitura rápida em palco.

| Token        | Família        | Peso | Tamanho | Line-height | Uso                                     |
|--------------|----------------|------|---------|-------------|-----------------------------------------|
| display      | Space Grotesk  | 700  | 32sp    | 38sp        | Hero, onboarding e estados vazios       |
| h1           | Space Grotesk  | 600  | 24sp    | 30sp        | Título de tela                          |
| h2           | Space Grotesk  | 600  | 20sp    | 26sp        | Seções e grupos                         |
| title        | Inter          | 600  | 18sp    | 24sp        | Preset, efeito e card                   |
| body         | Inter          | 400  | 16sp    | 24sp        | Texto operacional                       |
| body/compact | Inter          | 400  | 14sp    | 20sp        | Texto secundário                        |
| label        | Inter          | 600  | 12sp    | 16sp        | Labels de controle                      |
| technical    | JetBrains Mono | 500  | 12sp    | 16sp        | Sample rate, buffer, CPU, latência      |
| value        | JetBrains Mono | 500  | 14sp    | 18sp        | Valores de knobs, dB, BPM e percentuais |

**Regras:** não usar fonte monoespaçada em parágrafos; preservar no mínimo 14sp para informações que precisam ser lidas rapidamente; usar numerais tabulares em meters/telemetria; reservar caixa alta para labels pequenos. Se o tamanho do APK for prioritário, Inter pode assumir também os títulos.

**Distribuição:** o kit não inclui binários de fontes. Ao incorporar Space Grotesk, Inter ou JetBrains Mono no aplicativo, obtenha os arquivos pelas distribuições oficiais e mantenha as respectivas licenças de fonte no projeto.

## 2.2 Iconografia

**Estilo-base:** viewBox 24x24, stroke 1.8, linecap e linejoin arredondados, sem preenchimento por padrão e cor herdada do tema. O set deve parecer técnico e funcional, não ilustrativo.

| Categoria          | Tamanho visual | Touch target | Cor/estado       | Exemplos                                |
|--------------------|----------------|--------------|------------------|-----------------------------------------|
| Ação compacta      | 20dp           | 48dp         | text/secondary   | back, chevron, overflow, search         |
| Navegação          | 24dp           | 48-56dp      | primary/active   | presets, editor, footswitch, settings   |
| Bloco DSP          | 28dp           | 48dp         | accent semântico | drive, NAM, IR, EQ, delay, reverb       |
| Hero / empty state | 32dp+          | 48dp+        | brand/semantic   | MIDI disconnected, no preset, no device |

**Estados:** ativo = accent semântico + borda/surface ativa; bypass = iconografia e card dessaturados, mas ainda legíveis; disabled = opacidade reduzida e sem ação. Não depender exclusivamente de cor para comunicar estado.

**Ícones entregues no kit:** navegação (Presets, Editor, Footswitch, Settings), sistema (folder, search, power/bypass, drag, Bluetooth, USB, MIDI, input/output, tuner, tap tempo, info/link) e famílias DSP (Drive, NAM, IR, EQ, Compressor, Modulation, Delay, Reverb, Gate e Utility). Todos em SVG importável como Vector Asset no Android Studio.

**Regra semântica:** brand/NAM usa laranja; DSP algorítmico ativo usa teal; IR/cabinet usa amarelo; shaping/modulation usa roxo; efeitos temporais usam azul; error/clip usa vermelho apenas para alerta real.

### 3. Grid, espaçamento e dimensão

- Base grid: 4dp. Espaçamentos principais: 8, 12, 16, 24 e 32dp.
- Touch target mínimo: 48x48dp. Em telas de performance, preferir 56dp.
- Cards: raio de 12dp; inputs internos: 8dp; chips: 999dp.
- Margem horizontal padrão: 16dp no celular, 24dp no tablet.
- Knobs: 72-88dp em editor principal; 56-64dp em layouts compactos.

### 4. Arquitetura de UI recomendada

Stack sugerida: Kotlin + Jetpack Compose para a UI, ViewModel + StateFlow para estado observável e uma camada Kotlin/JNI para comandar a engine C++ de áudio. O processamento DSP permanece completamente fora do Compose.

Compose UI

- > ViewModel / UiState
- > DspController (Kotlin)
- > JNI bridge
- > AudioEngine (C++)
- > SignalGraph
- > Algorithmic DSP / NAM / IR

Regra crítica: parâmetros devem ser aplicados à engine por estruturas lock-free, atomics ou message queues apropriadas. Nunca executar carregamento de NAM/IR, I/O, parsing de preset ou alocação pesada dentro do callback de áudio.

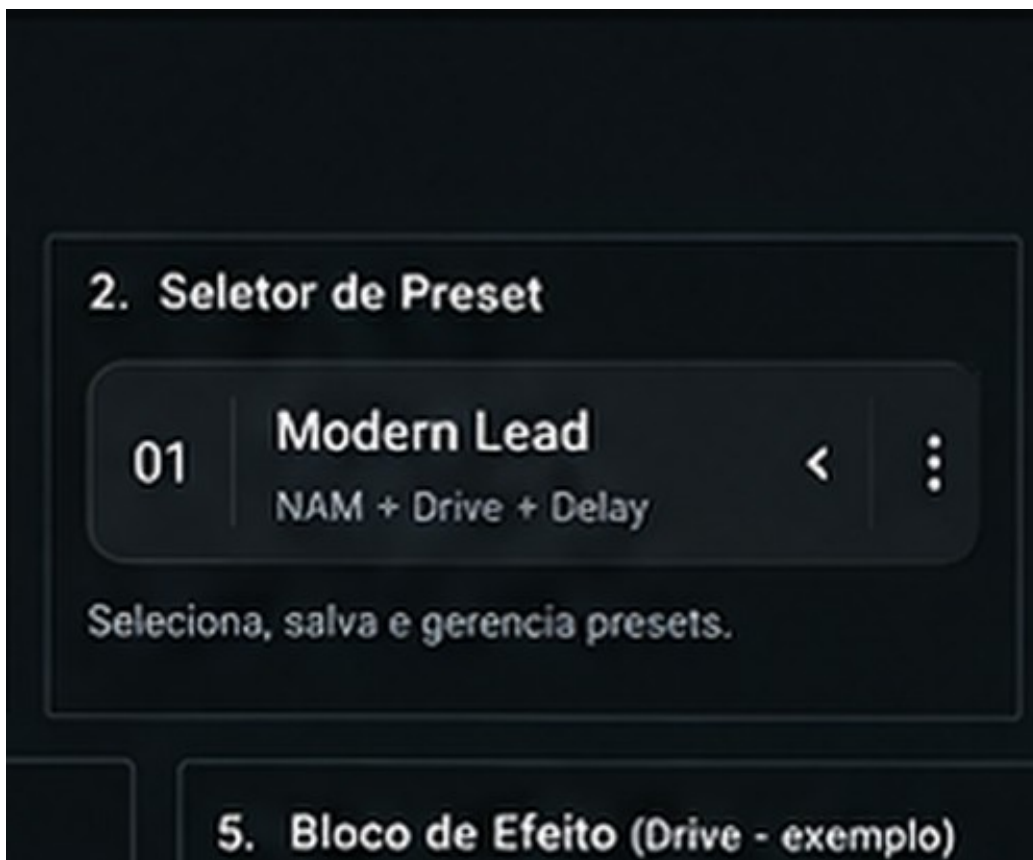
## 5. Componentes

### Top Bar



Marca, acesso rápido a arquivos/presets e menu. Exibe apenas ações globais. Altura recomendada: 56-64dp.

## Preset Selector



Mostra índice, nome e descrição curta do preset. Swipe horizontal pode alternar presets; toque abre picker.

## Signal Chain



Elemento central. Blocos ordenáveis, bypass visível e seleção por toque. Drag & drop deve ser opcional e nunca exigir precisão excessiva.

## NAM Effect Card



Editor para modelo neural. Deve exibir nome do modelo, bypass e parâmetros pós/pré-modelo quando existirem.

## Algorithmic Drive Card



Exemplo de processor ChowDSP-inspired. Controles DRIVE / TONE / LEVEL e seleção de algoritmo.

## IR Card



Seleciona IR/cabinet e oferece low cut, high cut e mix. Carregamento deve ocorrer em worker thread.

## EQ Card



Editor gráfico opcional + controles numéricos. Gestos devem ter alternativa acessível via sliders/inputs.

## Delay Card



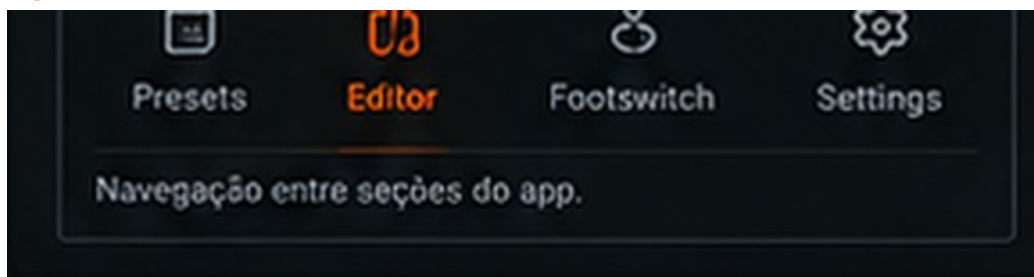
Tempo, feedback, mix e subdivision. Sincronização BPM pode ser adicionada como estado do processador.

## Input / Output Meters



Meters devem usar dados agregados enviados pela engine; nunca redesenhar diretamente na cadência do callback de áudio.

## Bottom Navigation



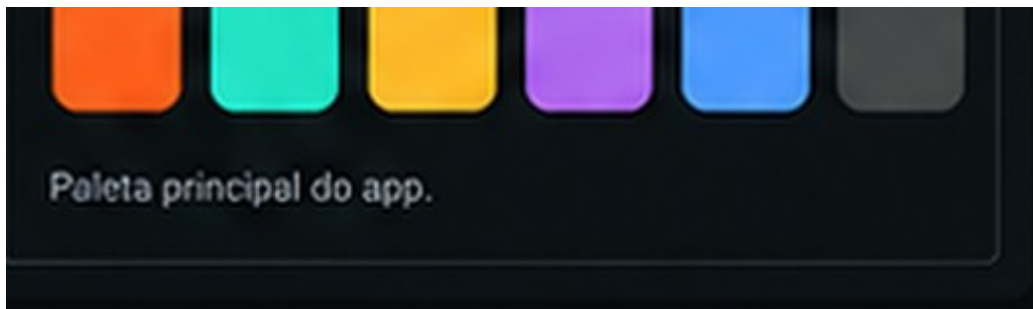
Quatro áreas: Presets, Editor, Footswitch, Settings. Manter estável em todas as telas principais.

## Footcontroller Status



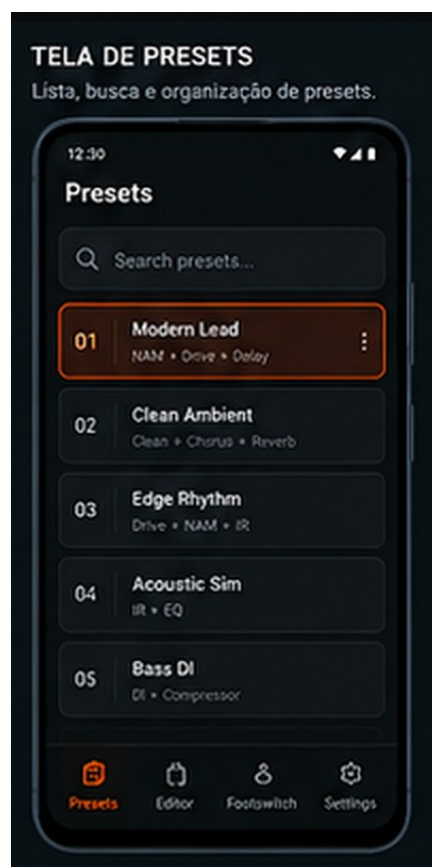
Mostra MIDI/Bluetooth/USB e estado Connected/Disconnected. Toque leva à tela de mapeamento.

## Accent Palette



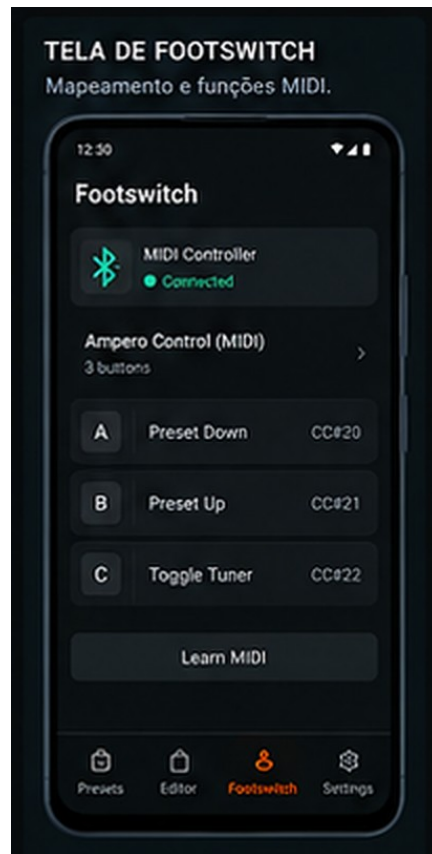
Cores são semânticas por família de DSP, não apenas decoração.

## Tela Presets



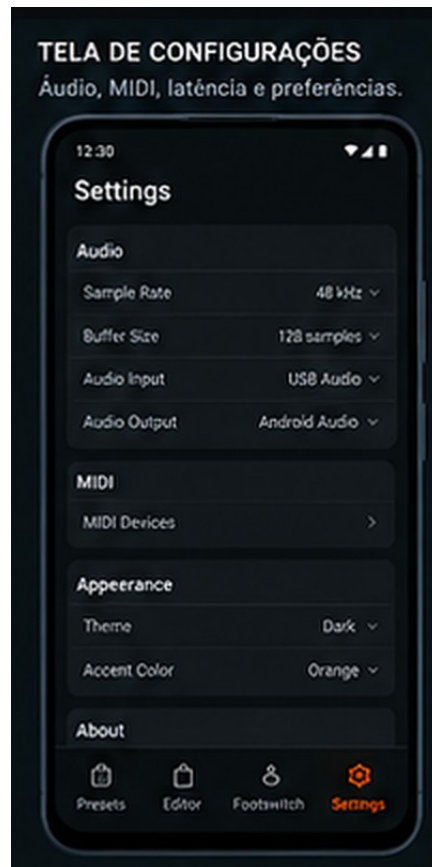
- Busca e ordenação locais.
- Favoritos e tags podem entrar depois sem alterar a navegação principal.
- Load de preset dispara transição segura na engine, preferencialmente com crossfade ou troca atômica de graph.

## Tela Footswitch



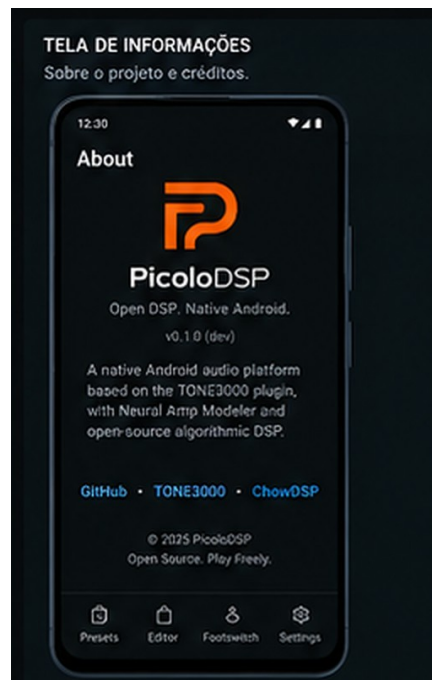
- Lista controllers detectados.
- Mapeamento por CC/PC e modo Learn MIDI.
- Ações mínimas: preset up/down, bypass, tuner, tap tempo e scene/snapshot quando suportado.

## Tela Settings



- Sample rate e buffer como informações/controles técnicos.
- Mostrar latência estimada e dispositivo de I/O.
- Mudanças que exigem restart da engine devem indicar isso explicitamente.

## Tela About



- Créditos e licenças de TONE3000, NAM e bibliotecas DSP.
- Versão do app e versão da engine.
- Links do repositório, documentação e licença.

## 6. Modelo de estado

```
data class DspUiState(
    val preset: PresetUi,
    val chain: List<EffectBlockUi>,
    val selectedBlockId: String?,
    val inputMeterDb: Float,
    val outputMeterDb: Float,
    val midi: MidiUiState,
    val engine: EngineUiState
)
```

O ViewModel expõe um estado imutável para o Compose. A engine não conhece UI. Eventos da UI são convertidos em comandos de domínio como SetParameter, ToggleBypass, MoveBlock, LoadNamModel, LoadIr e LoadPreset.

## 7. Signal graph e parâmetros

- Cada EffectBlock possui id estável, type, enabled, parameters e payload específico.
- A UI não deve depender da classe concreta C++ do efeito. Expor metadata suficiente para renderizar nome, parâmetros, ranges, units e agrupamentos.
- Normalizar parâmetros internamente em 0..1 quando possível, mantendo conversão para unidade de exibição na camada de metadata.
- Parâmetros contínuos devem usar smoothing na engine para evitar zipper noise.
- Bypass deve ser por bloco. Nunca reutilizar um estado global de bypass para NAM/IR/outros processors.

## 8. Contrato sugerido entre Kotlin e C++

```
setParameter(blockId, parameterId, normalizedValue)
setBypass(blockId, enabled)
moveBlock(blockId, newIndex)
loadNam(blockId, filePath)
loadIr(blockId, filePath)
loadPreset(presetJson)
getEngineSnapshot()
observeTelemetry() // meters, xruns, CPU, latency
```

Para comandos frequentes como knob movement, evite JNI excessivamente verboso. Uma fila de comandos compacta ou ponte com buffers/atomics reduz overhead. Telemetria de meters pode ser publicada em 20-30 Hz; a UI não precisa receber dados a cada bloco de áudio.

## 9. Interações críticas

- Knob: arraste vertical como gesto principal; double tap reseta para default; long press abre valor numérico.
- Bypass: botão no cabeçalho do bloco; mudança visual instantânea e comando assíncrono para engine.
- Reorder: long press + drag; durante o drag, não reconstruir a engine a cada pixel. Aplicar a nova ordem no drop.
- Preset change: preparar novo graph fora da audio thread e trocar de forma segura. Adicionar fade curto se necessário para evitar pop.
- NAM/IR load: exibir Loading e Error. Não bloquear a tela; manter o graph anterior até o novo recurso estar pronto.

## 10. Footcontroller / MIDI

A tela Footswitch deve tratar o controlador como uma extensão da interface. O mapeamento pode usar uma entidade MidiBinding(action, channel, messageType, number, min, max). A UI deve permitir Learn MIDI e mostrar feedback de evento recebido.

- Preset previous / next
- Toggle block bypass
- Select block
- Tap tempo
- Tuner
- Global mute
- Snapshot/scene futura

## 11. Performance e áudio

- Audio callback: zero I/O, zero locks bloqueantes, zero JSON, zero logging síncrono.
- Pré-alocar buffers e objetos usados em tempo real.
- Usar parameter smoothing em ganho, mix, filtros e outros parâmetros suscetíveis a click/zipper noise.
- Monitorar xruns/underruns e expor contador em Settings/diagnostics.
- Throttle de meters e telemetria para aproximadamente 20-30 updates/s.
- Carregamento de NAM e IR em worker thread; publicação do processor pronto por troca segura de referência/graph.

## 12. Roadmap de implementação da UI

| Fase   | Entrega                                                          |
|--------|------------------------------------------------------------------|
| Fase 1 | Design tokens + Theme Compose + BottomNavigation + TopBar.       |
| Fase 2 | PresetSelector + SignalChain estática conectada ao estado atual. |
| Fase 3 | EffectCard genérico + Knob + ToggleBypass + Parameter metadata.  |
| Fase 4 | NAM e IR loaders com estados Loading/Ready/Error.                |
| Fase 5 | Meters e telemetria de engine.                                   |

|               |                                                                   |
|---------------|-------------------------------------------------------------------|
| <b>Fase 6</b> | Preset browser e serialização completa da chain.                  |
| <b>Fase 7</b> | MIDI/footswitch + Learn MIDI.                                     |
| <b>Fase 8</b> | Tablet layout adaptativo, acessibilidade e testes de performance. |

## 13. Estrutura de pacote sugerida

```

app/
  ui/theme/
  ui/components/
  ui/screens/editor/
  ui/screens/presets/
  ui/screens/footswitch/
  ui/screens/settings/
  domain/model/
  domain/command/
  data/preset/
  engine/DspController.kt
  engine/jni/
cpp/
  audio/
  graph/
  processors/nam/
  processors/ir/
  processors/drive/
  processors/eq/
  processors/delay/
  midi/

```

## 14. Critério de pronto

- Todos os controles funcionam com toque e footcontroller quando aplicável.
- Bypass individual não afeta blocos adjacentes.
- Mudanças de parâmetros não introduzem pops perceptíveis.
- Troca de preset não executa trabalho pesado na audio thread.
- UI mantém 60 fps enquanto a engine processa áudio.
- Componentes são reutilizáveis entre celular e tablet.
- Licenças e atribuições de upstream estão acessíveis na tela About.

## 15. Handoff de assets

Estrutura do pacote v1.1:

```

components/
icons/effects/
icons/navigation/
icons/system/
typography/
tokens/

```

implementation/  
docs/

**components/** contém as referências visuais isoladas. **icons/** contém SVGs e manifest.json. **typography/** documenta famílias, pesos e escala. **tokens/** contém design\_tokens.json. **implementation/** contém stubs Kotlin para manter os valores sincronizados com o app.

**Convenção de nomes:** snake\_case para arquivos de asset e nomes sem referência a cor no nome do ícone. A cor deve vir do tema/estado, não do arquivo SVG.

**Versionamento:** alterações de aparência que não quebram contrato incrementam minor; mudanças de nome, dimensão-base ou semântica de tokens devem ser tratadas como alteração incompatível no design system.

PiccoloDSP UI Kit - versão conceitual 1.1